

Workstream Live API Drill Report

WS-POL-001-16 Terminal Benchmark Reference Flow

Flow Research / Workstream Engineering

2026-07-09

Field	Value
Report status	PASS
Evidence class	Privacy-redacted live API evidence
Initiative	WS-POL-001 Submission Artifact Policy Foundation
Chunk	WS-POL-001-16 Terminal Benchmark live API drill
Review boundary	HTTP-visible lifecycle proof, no database inspection
Public redaction	Exact fixture ids, local ids, hashes, byte counts, source labels, credentials

Executive Summary

This report proves that the current Workstream project setup and submission intake lifecycle works through public/operator HTTP APIs using a Terminal Benchmark reference fixture. The drill used a local Workstream stack, Flow-compatible tokens, Celery-backed setup execution, and the OpenAI Agents SDK adapter for setup-agent execution.

The lifecycle reached the intended final state:

```
ProjectGuide
-> GuideSourceSnapshot
-> GuideSufficiencyReport
-> SubmissionArtifactPolicy
-> EffectiveProjectSubmissionArtifactPolicy
-> project PreSubmitCheckerPolicy
-> task locked context
-> deterministic pre-submit
-> submission finalization
-> durable checker run
-> review_pending
```

The drill specifically proved four controls that were previously hard to see without database inspection:

Control	Result
Automatic setup pipeline	Setup moved from queued to policy_draft_ready through Celery and setup agents.
Project-scoped checker authority	One compiled project PreSubmitCheckerPolicy was locked and reused by the task.
Blocked intake side effect	Failed submission creation after pre-submit failure returned pre_submission_checker_failed and created no submission.
Durable pre-review gate	Finalization created a checker run and moved the task to review_pending.

Evidence Boundary

This PDF is the shareable evidence report. The corresponding Markdown evidence index records the PDF hash and validation outcomes; the internal review evidence and PR trust bundle record the exact local validation commands. The previous raw transcript was intentionally replaced because it was too large and too difficult to review safely.

The report does not include bearer tokens, API keys, raw local filesystem paths, raw database UUIDs, exact source-material hashes, source artifact byte counts, or source-specific task identifiers. Redaction placeholders are evidence labels, not replayable API literals.

The Terminal Benchmark fixture is used as a reference workload to test Workstream. Workstream is not becoming a Terminal Benchmark product fork, and no private source workflow is represented as a Workstream-owned product contract.

System Under Test

Component	Runtime used in drill
API	FastAPI on local loopback
Database	Local Postgres test database
Queue	Redis plus Celery worker
Auth	Flow-compatible HMAC tokens
Setup agents	OpenAI Agents SDK adapter
Proof method	API request/response observations

Database access was used only for migration reset before the drill. Lifecycle assertions in this report come from HTTP responses.

Source Material Snapshot

The project guide was created from a privacy-clean source snapshot bundle. The bundle represented the project guide, reviewer program material, task metadata, review packet material, static-check evidence, build evidence, and test evidence.

Snapshot item	Source kind	Sanitized durable reference
source-item-1	project_guide	public-fixture://source-item-1
source-item-2	reviewer_program	public-fixture://source-item-2
source-item-3	task_material	public-fixture://source-item-3
source-item-4	review_packet	public-fixture://source-item-4
source-item-5	checker_evidence	public-fixture://source-item-5
source-item-6	checker_evidence	public-fixture://source-item-6
source-item-7	checker_evidence	public-fixture://source-item-7
source-item-8	checker_evidence	public-fixture://source-item-8

The durable references above are public-safe placeholders. They prove that the snapshot used multiple named source items without exposing local paths, customer/source-system identifiers, or replayable object-storage locators.

The guide source snapshot produced:

```
guide_version: v1
source_snapshot_id: <redacted-id>
source_snapshot_hash: sha256:<redacted>
```

Setup Pipeline Evidence

After guide creation, Workstream automatically started the setup pipeline. The setup run advanced through the expected asynchronous states:

Phase	Observed status
Queue admission	queued
Guide sufficiency	running_sufficiency_agent
Policy derivation	running_policy_derivation_agent
Draft policy ready	policy_draft_ready

The sufficiency agent received the guide source material envelope, including the guide version, source snapshot id, source snapshot hash, and source items. It returned:

```
agent_name: ProjectGuideSufficiencyAgent
status: passed
source_snapshot_hash: sha256:<redacted>
```

The submission artifact policy derivation agent received the same guide source material plus the sufficiency report. It returned an agent-derived SubmissionArtifactPolicy draft for Workstream review:

```
derivation_source: agent_derivation
policy_hash: sha256:<redacted>
```

Draft and Approved Submission Artifact Policy

The submission-policy derivation agent produced a draft intake contract from the guide source snapshot. A project manager then approved an exact project policy for this drill. The effective policy and compiled checker came from that approved exact policy plus Workstream defaults, not from an unreviewed agent draft.

The agent-derived draft proposed project-level artifact classes such as task metadata, environment definition, rubric material, and verifier evidence. The approved exact policy bound those classes to the following public-safe artifact and evidence names:

Approved required artifacts, shown as public-safe display names:

Required artifact
submission archive
task metadata
static guard output
review packet
container build evidence
verifier run evidence A
verifier run evidence B

Approved required evidence, shown as public-safe display names:

Required evidence
task configuration
platform static guard output
automated review packet
docker build log
verifier execution log A
verifier execution log B

Required attestation terms:

Attestation term
all_or_nothing_reward
confidentiality and sensitive-data exclusion
container base images digest pinned
credentials and secret exclusion
dependencies pinned
SHA-256 hashes
human accountability for agent-assisted work
complete manifest
no generated caches
no reviewer-only material
no sensitive local material
offline verifier dependencies
original work
task layout matches metadata

Compiled Project Checker

The approved exact project policy was merged with Workstream defaults into an EffectiveProjectSubmissionArtifactPolicy. Workstream then compiled the deterministic project PreSubmitCheckerPolicy.

```
effective_policy_hash: sha256:<redacted>
compiled_bundle_hash: sha256:<redacted>
```

Compiled project checker bundle:

Checker	Purpose
check_submission_packet	Validate packet shape and required top-level data.
check_forbidden_files	Block files that must not enter the intake pipeline.
check_confidentiality_attestation	Require submitter accountability for sensitive-data exclusion.
check_required_files	Require project-specific files from the effective policy.
check_evidence_present	Require policy-derived evidence records.
check_evidence_integrity	Validate evidence structure and hash coverage.
check_low_quality_generated_artifacts	Warn or block obvious low-quality generated packets according to policy.

No task-specific checker was generated. The task locked references to the project guide snapshot, effective project submission artifact policy hash, and compiled project checker bundle hash.

API Lifecycle Index

The live drill used the following public/operator API sequence.

Step	API operation	HTTP
1	Create project	201
2	Create guide with source snapshot and policies	201
3	Poll setup run through queued, sufficiency, derivation, and ready states	200
4	Read sufficiency report	200
5	Read submission artifact policy	200
6	Approve submission artifact policy	200
7	Read effective submission artifact policy	200
8	Read project pre-submit checker policy	200
9	Activate guide	200
10	Create task	201
11	Screen task and lock policy context	200
12	Read locked context after screening	200
13	Release task	200
14	Activate worker profile	200
15	Claim task	200
16	Start task	200
17	Read locked context after start	200
18	Read worker work context	200
19	Read submission requirements	200
20	Run intentionally blocked pre-submit check	200
21	Attempt blocked submission create	422
22	Confirm no submission exists after blocked create	200
23	Read audit trail after blocked create	200
24	Run successful pre-submit check	200
25	Confirm successful pre-submit alone created no submission	200
26	Create submission	201
27	Confirm submission list contains created submission	200
28	Confirm worker cannot finalize submission	403

Step	API operation	HTTP
29	Finalize submission as project manager	200
30	Read finalized submission	200
31	List checker runs for submission	200
32	Read checker run details	200
33	Read final audit trail	200
34	Read final task state	200

Task Locking Evidence

The task was screened into the worker pipeline only after the project guide and policy context were ready.

Locked context after screening:

```
locked_guide_version: v1
locked_guide_source_snapshot_id: <redacted-id>
locked_guide_source_snapshot_hash: sha256:<redacted>
locked_effective_project_submission_artifact_policy_hash: sha256:<redacted>
locked_pre_submit_checker_bundle_hash: sha256:<redacted>
locked_review_policy_version: v1
locked_revision_policy_version: v1
locked_payment_policy_version: v1
```

The worker work context later reported:

```
status: in_progress
assigned_to_current_actor: true
can_run_pre_submit_check: true
can_submit: true
```

Blocked Intake Proof

The blocked packet intentionally omitted the static guard artifact. Because the approved exact policy also requires that source item as evidence, the same payload failed both the required-file and required-evidence gates.

Pre-submit response:

```
{
  "task_id": "api-drill-task",
  "authoritative": false,
  "status": "failed",
  "eligible_to_submit": false,
  "results": [
    {
      "checker_name": "check_required_files",
      "status": "failed",
      "severity": "high",
      "would_block_if_submitted": true,
      "worker_message": "Required artifact is missing.",
      "worker_suggested_fix": "Add the required artifact and rerun pre-submit.",
    }
  ]
}
```

```

        "worker_evidence_refs": []
    },
    {
        "checker_name": "check_evidence_present",
        "status": "failed",
        "severity": "high",
        "would_block_if_submitted": true,
        "worker_message": "Required evidence is missing.",
        "worker_suggested_fix": "Attach the required evidence and rerun pre-submit.",
        "worker_evidence_refs": []
    },
    {
        "checker_name": "check_submission_packet",
        "status": "passed",
        "severity": "info",
        "would_block_if_submitted": false,
        "worker_message": "Submission packet shape is valid.",
        "worker_suggested_fix": null,
        "worker_evidence_refs": []
    }
]
}

```

Submission creation response:

```

{
  "code": "pre_submission_checker_failed",
  "details": {
    "task_id": "api-drill-task",
    "authoritative": false,
    "status": "failed",
    "eligible_to_submit": false,
    "results": [
      {
        "checker_name": "check_required_files",
        "status": "failed",
        "severity": "high",
        "would_block_if_submitted": true,
        "worker_message": "Required artifact is missing.",
        "worker_suggested_fix": "Add the required artifact and rerun pre-submit.",
        "worker_evidence_refs": []
      },
      {
        "checker_name": "check_evidence_present",
        "status": "failed",
        "severity": "high",
        "would_block_if_submitted": true,
        "worker_message": "Required evidence is missing.",
        "worker_suggested_fix": "Attach the required evidence and rerun pre-submit.",
        "worker_evidence_refs": []
      }
    ]
  }
}

```

No-side-effect proof:

Check	Result
Task submission list after blocked create	Empty
Audit event pre_submission_check_failed	Present
Audit event submission_created	Absent
Audit event pre_review_gate_started	Absent

Checker-run list/get APIs are submission-scoped. Before a submission id exists, there is no valid checker-run endpoint for the blocked intake proof; the HTTP-visible proof is the empty submission list plus task audit events.

This confirms that failed pre-submit intake is not a product review decision. It blocks submission creation before a review packet exists.

Successful Submission Proof

The successful packet passed all non-authoritative pre-submit checks:

Checker	Result
check_submission_packet	passed
check_forbidden_files	passed
check_confidentiality_attestation	passed
check_required_files	passed
check_evidence_present	passed
check_evidence_integrity	passed
check_low_quality_generated_artifacts	passed

The successful pre-submit response preserved the preflight contract:

```
{
  "task_id": "api-drill-task",
  "authoritative": false,
  "status": "passed",
  "eligible_to_submit": true,
  "results": [
    {
      "checker_name": "check_submission_packet",
      "status": "passed",
      "severity": "info",
      "would_block_if_submitted": false,
      "worker_message": "Submission packet shape is valid.",
      "worker_suggested_fix": null,
      "worker_evidence_refs": []
    },
    {
      "checker_name": "check_required_files",
      "status": "passed",
      "severity": "info",
```

```
      "would_block_if_submitted": false,  
      "worker_message": "Required artifacts are present.",  
      "worker_suggested_fix": null,  
      "worker_evidence_refs": ["artifact-manifest"]  
    }  
  ]  
}
```

The successful pre-submit call did not create a submission by itself. Submission creation was a separate API call and returned:

```
HTTP: 201  
submission_version: 1  
submission_id: <redacted-id>
```

The worker finalize attempt correctly failed:

```
HTTP: 403  
detail: actor lacks required role
```

Project-manager finalization succeeded and started the pre-review gate:

```
HTTP: 200  
finalized_at: present
```

Durable Checker Run Evidence

After finalization, Workstream created a durable checker run for the submission. Checker-run list and get both returned:

```
status: completed  
routing_recommendation: allow_review  
passed_count: 8  
warning_count: 0  
failed_count: 0  
blocking_count: 0  
triggered_by: workstream-system:pre-review-gate
```

Durable checker results:

Checker	Result
check_submission_packet	passed
check_policy_context_present	passed
check_evidence_present	passed
check_evidence_integrity	passed
check_required_files	passed
check_forbidden_files	passed
check_confidentiality_attestation	passed
check_low_quality_generated_artifacts	passed

Audit Trail

The final task audit event sequence was:

Event	State transition
task_created	null -> draft
task_status_changed	draft -> screening
task_status_changed	screening -> ready
task_status_changed	ready -> claimed
task_status_changed	claimed -> in_progress
pre_submission_check_failed	in_progress -> in_progress
submission_created	in_progress -> submitted
submission_finalized	submitted -> submitted
pre_review_gate_started	submitted -> evaluation_pending
pre_review_gate_passed	evaluation_pending -> review_pending

The final task response confirmed:

```
status: review_pending
locked_guide_source_snapshot_hash: sha256:<redacted>
locked_effective_project_submission_artifact_policy_hash: sha256:<redacted>
locked_pre_submit_checker_bundle_hash: sha256:<redacted>
```

Verification Summary

Local validation passed before this report was prepared.

Verification	Result
Stale wording scan	PASS
Markdown link check	PASS
Focused backend tests	PASS
API contract drill	PASS
Terminal Benchmark example lint	PASS
Terminal Benchmark example bytecode compile	PASS
Public-safe failure output check	PASS
Targeted privacy scan	PASS, only intentional backend test literals remained
Diff whitespace check	PASS

Internal Review Summary

Required internal reviewers passed after the privacy scrub.

Reviewer track	Result
Senior engineering	PASS
QA/test	PASS WITH LOW RISKS
Security/auth	PASS
Product/ops	PASS WITH LOW RISKS
Architecture	PASS WITH LOW RISKS
Docs	PASS WITH LOW RISKS
Reuse/dedup	PASS
Test delta	PASS WITH LOW RISKS
CI integrity	PASS WITH LOW RISKS

Final Assessment

The live API drill proves the current Workstream v0.1 setup and intake spine for this reference project:

- Project setup ran asynchronously and produced a sufficiency report plus derived submission policy.
- The effective project policy compiled into a deterministic project checker bundle.

- The task locked the active guide, policy, checker, review, revision, and payment context.
- Failed pre-submit did not create a submission.
- Successful submission creation and finalization created a durable checker run.
- The final task reached `review_pending` through the automated pre-review gate.

The evidence supports closing this chunk at the human checkpoint for PR #84. The next L1 chunk should not begin until the user explicitly approves that transition.